C FOR PEBBLE PROGRAMMERS - EMERY (2026)

C for Pebble Programmers - Emery syllabus

Platform: Pebble Time 2 (emery, 200 x 228 rectangular display)
Language: C with the Pebble SDK
Activities: 21, grouped into seven core skills and four phases
Total estimated focused time: 19 hours 15 minutes (actual time may vary)

Purpose and approach

Learn Pebble development by reading, modifying, testing, and eventually remixing small, working C examples. Activities are evidence-based: an activity is complete only when its observable result or learning artifact satisfies the acceptance criteria and you can independently explain how it works.

No AI during coursework. Do the design, coding, research, troubleshooting, and explanations yourself using SDK documentation, the included Emery examples (https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/examples/README.md), compiler diagnostics, logs, emulator, and Git.

Seven core skills

1. UI Design - typography, layout, readable screens and interaction states.
2. Idea Generation - possible applications, creative alternatives and scoped concepts.
3. Algorithmic Thinking - sequencing, state, pseudocode and transitions.
4. C Programming - syntax, functions, pointers, callbacks and reusable code.
5. Resource Literacy - finding and interpreting SDK documentation and examples.
6. Debugging and Testing - reproducing failures, tracing causes, verifying fixes.
7. Development Tools and Workflow - SDK builds, Emery emulator, Git and integration.

Completion standard

For every activity: (a) meet the Done when criteria, (b) preserve a verifiable artifact such as source code, a successful build, sketches, pseudocode, logs, or a bug report, and (c) be able to explain your own work. Meeting a technical objective does not require choosing a specific aesthetic or app idea.

Activity IDs and difficulty

The digit indicates the core skill; A/B/C is low/medium/high within that skill. Tasks are taught in dependency order, not strict numeric order. Priority H/M/L is a planning cue, not a grade. Times are focused-work estimates, not deadlines.

Local example library

Start with lessons/README.md (https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/lessons/README.md). All assigned starter code in this syllabus lives in this repository; external reference links are secondary background.

Phase I: Orientation and first modification

2A Explore Pebble examples

Skill: Idea Generation | Estimate: 20m | Difficulty: Low | Priority: High | Prerequisites:

None

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/2A

Do: Browse only the included Emery-targeted projects under examples/, initially time-only, font-selector, and persist-counter. Choose one and brainstorm five creative variations; describe what changes and what new API each might require. Do not browse or solve the debugging challenges yet.

Done when: Five distinct ideas are recorded, each describes a possible change, and one idea is marked as the most interesting to investigate.

7A Build and run a time-only C watchface

Skill: Development Tools and Workflow | Estimate: 30m | Difficulty: Low | Priority: High | Prerequisites: None

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/7A

Do: Create a new C project with the CURRENT watchface tutorial for SDK 4. Set package.json targetPlatforms to ["emery"]. Follow the tutorial through Displaying the Time and STOP before Adding the Date. Run pebble build then pebble install --emulator emery; confirm time updates and only one TextLayer exists. Identify project structure and build output.

Done when: The project builds and runs in the emulator, displays the current time, updates by minute, and has no date TextLayer.

4A Annotate the time-only watchface code

Skill: C Programming | Estimate: 45m | Difficulty: Low | Priority: High | Prerequisites: 7A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/4A

Do: Use your time-only watchface source. In your own notes, annotate the variable declarations, data types, pointers, function calls, arguments, and callback registrations. For each function, state its responsibility. Mark which calls happen directly and which functions Pebble calls as callbacks. Do not use AI to explain or write the annotations.

Done when: Your notes explain the key declarations and each function’s role, and distinguish direct calls from callbacks.

5A Find and document a Pebble API function

Skill: Resource Literacy | Estimate: 20m | Difficulty: Low | Priority: High | Prerequisites: 4A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/5A

Do: Find the SDK documentation for text_layer_set_font() or another function used by your watchface. Record what it does, its full function signature, the type and purpose of each argument, its return type, and one example of where it is used. Write the explanation in

your own words.

Done when: A concise API note contains purpose, signature, argument meanings, return type, and a usage example.

3A Trace the watchface lifecycle

Skill: Algorithmic Thinking | Estimate: 30m | Difficulty: Low | Priority: High | Prerequisites: 4A, 7A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/3A

Do: Draw the execution flow for the time-only watchface. Include main(), init(), window_stack_push(), main_window_load(), update_time(), tick_handler(), app_event_loop(), main_window_unload(), and deinit() where applicable. Show which functions call other functions and which are callbacks. Show the separate path for a minute tick after startup.

Done when: A flow diagram shows startup, initial display, minute updates, and cleanup, with direct calls distinguished from callbacks.

1A Change one visual property of a time-only watchface

Skill: UI Design | Estimate: 30m | Difficulty: Low | Priority: High | Prerequisites: 4A, 5A, 7A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/1A

Do: Start from the time-only checkpoint of the C watchface tutorial: the code after 'Displaying the Time' and before 'Adding the Date'. Choose exactly one visual property to change: font, position, alignment, text color, or background color. Make the change, build, and inspect it in the emulator. Keep the time working and do not add the date or other functionality.

Done when: The app builds and runs, the time still updates, and one intentional visual change is visible. The watchface still has no date layer.

6A Diagnose a simple compiler error

Skill: Debugging and Testing | Estimate: 20m | Difficulty: Low | Priority: High | Prerequisites: 4A, 7A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/6A

Do: Start from the deliberately flawed examples/debug-6a project. Run pebble build; capture the first compiler diagnostic; identify and correct the single syntax fault; rebuild and run on emery. The original clean time-only example is for comparison AFTER the diagnosis.

Done when: You can show the original error, identify its cause, fix it, and produce a successful build.

Phase II: Reading, interaction, and remixing

7B Create a Git-based modification workflow

Skill: Development Tools and Workflow | Estimate: 45m | Difficulty: Medium | Priority: High | Prerequisites: 7A, 1A, 6A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/7B

Do: Create your own working copy of the example; do not edit the reference repository in place. Initialize or use a Git repository, commit the known-good starting point, make one small visual modification, build and test it, then commit the working change with a meaningful message. Confirm you can inspect the diff and return to the earlier version.

Done when: The repository has a baseline commit and a modification commit, the app builds, and you have inspected the diff or demonstrated how to restore the baseline.

5B Trace an existing feature through source code

Skill: Resource Literacy | Estimate: 45m | Difficulty: Medium | Priority: Medium | Prerequisites: 3A, 4A, 5A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/5B

Do: Open the Font Browser example. Find where button input is registered or handled, identify the state that changes, and follow the code until the display is updated. Use the README and SDK docs to look up unfamiliar APIs. Write down the file/function names and draw the input -> state change -> display update path.

Done when: Your notes identify the input handling, changed state, display update, and the relevant source locations and documentation.

1B Add and style a date TextLayer

Skill: UI Design | Estimate: 45m | Difficulty: Medium | Priority: Medium | Prerequisites: 1A, 4A, 5A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/1B

Do: From YOUR time-only emery code, follow the Adding the Date section of the current C tutorial to create, style, position, update, and unload a second TextLayer. Arrange time and date for the Emery 200x228 screen. Build and install on emery. Do not switch to a completed repository.

Done when: The watchface displays both time and date, both update correctly, and you can explain how the date layer is created, styled, attached, updated, and destroyed.

3B Design a font-cycling algorithm

Skill: Algorithmic Thinking | Estimate: 30m | Difficulty: Medium | Priority: Medium | Prerequisites: 3A, 5B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/les

sons/3B

Do: Before editing code, write pseudocode for cycling through a list of fonts in both directions. Define how the selected font is represented, what Up and Down do, whether the list wraps at either end, and how the screen is refreshed. Work through the first item, last item, and a one-item list. Keep this as an algorithm-design exercise; do not implement it yet.

Done when: Pseudocode and a small flowchart cover both directions, normal selection, and boundary cases.

2B Write a two-example remix brief

Skill: Idea Generation | Estimate: 30m | Difficulty: Medium | Priority: Medium | Prerequisites: 2A, 5B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/2B

Do: Select two examples from the included examples/ folder (such as time-only and persist-counter). Describe a possible new application using an element from each. State its purpose, the elements to borrow, the smallest working version and a technical uncertainty. Do not implement yet.

Done when: A one-page brief explains the purpose, source features, minimal version, and one or more technical questions to investigate.

4B Modify the Font Browser

Skill: C Programming | Estimate: 120m | Difficulty: Medium | Priority: Medium | Prerequisites: 3B, 5B, 7B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/4B

Do: Start from the working local Emery font-selector watchapp. Trace its input handler, index state, and apply_font function; add one extension you design yourself (e.g., index indicator or extra preview), committing and testing after each change. This is a watchapp, so Up/Down are available.

Done when: Your modified app builds and runs, the original font browsing still works, and your added feature is visible and tested.

6B Diagnose a font-selection boundary bug

Skill: Debugging and Testing | Estimate: 45m | Difficulty: Medium | Priority: Medium | Prerequisites: 4B, 6A

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/6B

Do: Use the provided deliberately flawed examples/debug-6b project. It compiles but navigation behaves incorrectly. Reproduce four successive Up and Down presses, collect the included APP_LOG index traces with pebble logs --emulator emery, locate the wrong transition, apply a minimal fix and verify both boundaries. Compare with the clean example

only after debugging.

Done when: The bug is reproducible, the cause is explained, and tests show that cycling works correctly at both ends of the list.

Phase III: Independent research and design

5C Research an unfamiliar Pebble capability

Skill: Resource Literacy | Estimate: 60m | Difficulty: High | Priority: Medium | Prerequisites: 5B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/5C

Do: Study the local Emery persist-counter app. Identify how it checks for saved data, reads it at startup, and writes changes. Research the SDK's persistence functions using official documentation, and write an implementation plan to reuse the pattern in another app; no copy-and-paste integration yet.

Done when: A technical note names the capability, links the documentation and example, lists relevant APIs and constraints, and proposes implementation steps.

2C Develop an original Pebble app concept

Skill: Idea Generation | Estimate: 60m | Difficulty: High | Priority: Medium | Prerequisites: 2B, 5C

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/2C

Do: Use the included examples/ collection as inspiration. Propose three original Pebble Time 2 applications based on your interests. Compare feasibility and learning value; write a minimal specification for one without committing to build it.

Done when: Three ideas are compared and one has a concise specification for purpose, interaction, minimum version, and technical uncertainty.

3C Design a state machine for an app

Skill: Algorithmic Thinking | Estimate: 60m | Difficulty: High | Priority: Medium | Prerequisites: 2C, 3B, 4B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/3C

Do: Use your original emery app concept or the Emery Touch Tone watchapp as an input-pattern reference. Enumerate states, transitions, button press behavior, screen changes, and edge cases. Create a state diagram and table. Plan for Emery input and display only.

Done when: A state diagram and transition table describe all main states, triggers, resulting states, and visible responses, including important edge cases.

1C Design a multi-mode Pebble interface

Skill: UI Design | Estimate: 90m | Difficulty: High | Priority: Medium | Prerequisites: 1B, 2C, 3C, 4B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/1C

Do: Using your concept from 2C and the state diagram from 3C, draw each interface mode for the Emery 200x228 rectangular screen. Use layouts from the included simple examples as starting references, but develop your own visual hierarchy and navigation.

Done when: A set of labeled sketches covers every main state, navigation between states, button outcomes, and screen-size considerations.

4C Refactor font selection into a reusable component

Skill: C Programming | Estimate: 90m | Difficulty: High | Priority: Medium | Prerequisites: 3B, 4B, 5C

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/4C

Do: Using your emery-compatible Font Browser implementation, refactor selected font state and update logic into reusable functions without duplicating cases. Use current C watchface custom font tutorial Part 2 if you add font resources. Confirm correct loading/unloading. Build, install, and test on emery exclusively.

Done when: Font selection uses a reusable function and organized data, both navigation directions work, and any allocated font resources are cleaned up correctly.

Phase IV: Integration and verification

7C Integrate font browsing with persistent selection

Skill: Development Tools and Workflow | Estimate: 150m | Difficulty: High | Priority: Medium | Prerequisites: 2C, 4C, 5B, 7B

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons/7C

Do: Start with the local clean font-selector project and the local persist-counter reference. Add persistent selection to your own font app, saving its font index and restoring it on startup, with a safe default for missing/out-of-range values. Build and test on Emery; do not simply copy the integrated debugging sample.

Done when: The combined app builds, font navigation works, the selected font survives a restart, invalid or absent stored values are handled safely, and Git records the work.

6C Diagnose a persistent-state integration bug

Skill: Debugging and Testing | Estimate: 90m | Difficulty: High | Priority: Low | Prerequisites: 4C, 6B, 7C

Starting example: https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/les

sons/6C

Do: Use the deliberately flawed examples/debug-6c project. Select a nondefault font, exit, reopen; document the discrepancy. Inspect the provided startup and selection logs; trace storage write/read behavior; fix the single persistence defect and test fresh install plus repeated restarts. Do not consult a clean integrated implementation before diagnosing.

Done when: A reproducible bug report records expected and actual behavior, the cause is explained, and the fix passes fresh-start, selection, and restart tests.

Dependencies and tracking

Real Taskwarrior depends: relationships are created by taskwarrior/import_course.py (https://github.com/Darachnid/c-for-pebble-programmers-course/blob/main/taskwarrior/import_course.py). After importing, use:

sh
task project:pebble.course list
task project:pebble.course unblocked
task <ID> info

The importer uses the course IDs to avoid duplicating already imported activities; course-generated annotations are refreshed to point to this repository.

Optional remix levels

1. Modify one detail in a single example.
2. Customize several details in one example.
3. Borrow one feature from another example.
4. Combine the functionality of two examples.
5. Combine selected functionality from three examples.
6. Assemble components into an application with an original purpose.

The 21 activities are the required core; later remix levels can be pursued independently.

Reference information

- Pebble SDK documentation (https://developer.repebble.com/docs/c/)
- Official C watchface tutorial (https://developer.repebble.com/tutorials/watchface-tutorial/part1/)
- Persistent-storage API guide (https://developer.repebble.com/guides/events-and-services/persistent-storage/)

All examples were written specifically for this course and are Emery-targeted. A local SDK build has not been run by the course publisher; always verify in your own emulator.

Self-contained lesson starters

Each activity now has an exact starter folder at lessons/<ID>/ (https://github.com/Darachnid/c-for-pebble-programmers-course/tree/main/lessons) with its own C source, manifest, build script and lesson instructions. Relevant licensed SDK source samples are copied into the lesson folder, and their license is retained. The local examples are authoritative; external links provide optional background rather than required source

material.